

Automation: Desktop Flow (Advanced)

Power CAT

Module 6: RPA Advanced

Overview

The **Northwind User Admin Portal** — the legacy web application where Northwind Traders' partner user accounts are created — predates the company's move to Dataverse: no API, no connector, just a sign-in form and a data-entry screen. In this module, you build a desktop flow with **Power Automate for desktop** that operates the portal the way a person would — signing in, reading partner contacts from a CSV file, validating every record, and typing the valid ones into the form — and you build it the way production automations are built: configurable inputs, reusable subflows, error handling, resilient selectors, and UI elements published for reuse.

This module also completes a decision rule the series has been building. If a connector or API exists, use it — that's Modules 1 and 2, where even the desktop flow talked to Dataverse through connector actions. If no API exists but the interface is predictable, script the UI step by step — that's this module: classic screen-level RPA, with saved references to each control. And when even the UI isn't predictable, describe the task instead of scripting it — that's [Module 4](#)'s computer use. The portal you automate here is stable and predictable, which makes it exactly RPA's territory.

If desktop flows are new to you — or you'd like a refresher on the designer, `%variable%` expressions, and the console before adding this module's structure on top — [Module 2](#) builds one from scratch.

By the end, you will have hands-on experience with the key building blocks of production-grade RPA: configurable inputs, subflows, data validation, error handling, retry logic, and UI collections.



Tip: This lab runs as an attended desktop flow on your own Windows machine. Avoid using the keyboard or mouse while the flow is running to prevent interruptions.

Why this matters

Manual data entry tends to be slow and inconsistent, and it pulls skilled staff away from higher-value work. Copy-paste workflows can also introduce errors that often stay hidden until they lead to downstream problems. RPA handles these predictable, repetitive tasks reliably, giving people more time for other work.

Scenario

Northwind Traders onboards dozens of new staff every month. Each new hire must be registered in the **Northwind User Admin Portal** — a legacy web application that cannot be replaced or integrated with directly. Today, an administrator manually copies each person's details from a CSV file into the portal form, one row at a time. The process is slow, error-prone, and blocks the team from higher-value work.

The data itself is imperfect: names arrive with extra spaces, phone numbers are sometimes too short, and email addresses occasionally lack a valid domain structure. Submitting bad data causes silent failures that are only discovered later, requiring rework.

Your goal is to eliminate this manual process by building the **Northwind User Entry Automation** — a desktop flow that:

- Signs in to the portal securely using configurable credentials.
- Reads the source CSV and validates every record before touching the portal.
- Skips invalid rows and logs the reason, so nothing is lost silently.
- Submits each valid record and confirms the portal acknowledged the entry.
- Handles transient failures with retry logic so a single network hiccup does not abort the entire run.

By the end of this lab, the administrator's role shifts from doing the data entry to reviewing the execution log — a task that takes minutes instead of hours.

Core concepts

Concept	Why it matters
Desktop flow	an automation built in Power Automate for desktop that performs actions on a Windows machine.
UI element	a saved reference to a control on the screen, such as a button, field, or message.
Variable	a named value used to pass configuration, input data, and results through the flow.
Subflow	a reusable group of actions that keeps the automation organized and easier to test.
Validation	a rule that checks whether data is safe and complete before the flow uses it.
Error handling	actions that retry, redirect, log, or stop the flow when something unexpected happens.

Where RPA runs

Desktop flows run on Windows. Power Automate supports two runtime options — choose the one that fits your setup.



Option 1 — Bring your own machine

Use a Windows machine that you own and have registered in Power Automate.

- Install the latest **Power Automate for desktop** app.
- Install the **browser extension** required for web actions.
- Avoid using the keyboard or mouse while an attended flow is running.

Option 2 — Hosted Machines

Use Microsoft-hosted Windows machines that scale without physical hardware.

- Sign in with your work or school account to reach Microsoft 365, SharePoint, and Azure.
- Run attended or unattended desktop flows without maintaining your own machine.
- Choose a standard or custom VM image.
- Connect to your own virtual network.
- Join the machine to Microsoft Entra ID and enroll it in Intune for full governance.

 **For this lab** - This module uses your own registered Windows machine to automate the Northwind User Admin Portal.

Credentials and access control

Before you build, decide how the desktop flow signs in and which systems it is allowed to reach.

Credentials to use

- **Machine credentials** - the Windows credentials used to start and run the desktop flow. In production, retrieve them securely through an approved credential store.
- **Application credentials** - the username and password used by the Northwind User Admin Portal. Store production secrets in Power Automate credentials, Azure Key Vault, CyberArk, or another approved secret store.

Prerequisites

The scenario continues the earlier modules, but the build stands alone — you can complete this module without them. You need: - Review the [workshop prerequisites](#) before you begin. - A **Windows machine** with **Power Automate for desktop** installed. It comes preinstalled with Windows 11; on Windows 10, [install it from Microsoft](#). - **Microsoft Edge** with the **Power Automate browser extension** [installed and enabled](#) — the web automation actions in this module need it to reach the page's controls. If your organization's policies block browser extensions, you're not stuck: in the launch action, set **Browser interaction method** to **WebDriver** — an extension-free method. It needs a WebDriver executable matching your browser version in `%LocalAppData%\Microsoft\Power Automate Desktop\WebDrivers`, and it doesn't work where policy forces browser sign-in. - A **Power Automate license** that covers attended desktop flows — the RPA features used in this module, including the UI elements collection in Step 9. Licensing names and entitlements change over time, so this lab does not name a specific SKU. Review the [types of Power Automate licenses](#) to confirm the current requirement, and use the setup check below if you need a trial.

 ****Setup check:**** If Power Automate for desktop or the portal prompts you about premium features, select the option to ****start a free trial**** and continue with the lab.

- Access to the **Northwind User Admin Portal**. Before you start, confirm that <https://nwtua.z13.web.core.windows.net/> opens in Microsoft Edge and shows the portal's sign-in dialog.

i Before you continue

Complete each preflight task before you begin:

1. Open Power Automate for desktop and confirm that you can sign in.
2. Open the Northwind portal in Microsoft Edge and confirm that the page loads.
3. Ask the facilitator for the workshop username, password, and CSV file.
4. Close unrelated applications so they do not interrupt the attended run.

Learning objectives

By the end of this module, you will:

- Structure a desktop flow into numbered subflows with configurable input variables, and mark secrets as sensitive.
- Capture web UI elements and automate sign-in to a legacy web application.
- Validate CSV records with conditions and a regular expression before any data entry happens.
- Populate and submit a web form for each valid record, and log every outcome.
- Add retry policies, error paths, and self-healing, and harden a selector against a dynamically generated ID.
- Publish UI elements as a reusable collection that other desktop flows can share.

How to use this lab

Work through the steps in order. Each section builds on the variables, UI elements, and Subflow created earlier.

i Screenshots and product updates The screenshots are illustrative and may differ slightly from your version of Power Automate for desktop. If the interface differs, use the written action name, variable name, and expected result to identify the equivalent control.

Complete one action at a time and compare your screen with the screenshot before moving on.

Save the desktop flow after each checkpoint so you always have a working known-good recovery point.

If a label or button has moved, use the action name and the surrounding description, instructions to locate the equivalent control.

The lab uses a consistent naming convention: the `f_` prefix marks variables the flow produces, and the `ci_` prefix — *configurable input* — marks values supplied to the flow from outside. Each name is introduced the first time it is used. For a full reference, see [Appendix J: Names used in this lab](#).

Documentation & learning resources

- [Power Automate for desktop documentation](#)
- [UI elements in Power Automate for desktop](#)
- [Variables in Power Automate for desktop](#)
- [Error handling in desktop flows](#)
- [UI collections \(reusable UI elements\)](#)
- [Hosted machines overview](#)

- [Create an Azure Key Vault credential](#)
- [Run a desktop flow in picture-in-picture](#)
- [Debugging a desktop flow](#)
- [Handle errors in desktop flows](#)
- [Safe stop](#)
- [Flow control actions](#)

Lab instructions

Step 1: Create the desktop flow and configure variables and validations

Step-by-step instructions



Goal - Create the desktop flow, define configurable inputs, and build prechecks.

1. Open **Power Automate for desktop**.

Opening Power Automate for desktop

Figure: Opening Power Automate for desktop.

1. Sign in with your organizational account.

Signing in with your organizational account

Figure: Signing in with your organizational account.

1. Confirm that the environment picker in the upper-right corner shows the developer environment you want to use.

Confirming the target environment in the environment picker

Figure: Confirming the target environment in the environment picker.

1. Select **New**, and then select **Flow**.

Creating a new flow

Figure: Creating a new flow.

1. Enter **Provision Partner Users** as the flow name, and then select **Create**.

Naming the desktop flow Provision Partner Users

Figure: Naming the desktop flow Provision Partner Users.

1. Create an **input variable** for the website URL so it can be changed without editing the flow. Select **Variables** located next to the workspace.

The desktop flow designer

Figure: The desktop flow designer.

1. In the **Variables** pane, select **Variables** and select the **plus icon**, and then select **Input**.

Adding an input variable from the Variables pane

Figure: Adding an input variable from the Variables pane.

1. Configure the **URL input variable** with the values below and click **Save**.

- **Variable name:** ci_NWUA_URL. The ci prefix means configurable input; NWUA means Northwind User Admin Portal.
- **Data type:** Text.
- **Default value:** <https://nwtua.z13.web.core.windows.net/>
- **External name:** ci_NWUA_URL. This is the name displayed in Power Automate and in the flow's run details.
- **Description:** Northwind User Admin Portal URL.

Configuring the ci_NWUA_URL input variable

Figure: Configuring the ci_NWUA_URL input variable.

1. Create the remaining portal and browser input variables using the values in the following table. Mark both the **username** and **password** variables as **sensitive** so their values are masked in the designer and run details.

Variable name	Data type	Default value	External name	Description
ci_NWUA_username	Text	master	ci_NWUA_username	Northwind User Admin Portal username
ci_NWUA_Password	Text	iAmReady	ci_NWUA_Password	Northwind User Admin Portal password
ci_Edge_Path	Text	C:\Files (x86).exe	ci_Edge_Path	Microsoft Edge executable path

 **Production note — do not ship hard-coded credentials:** This lab stores the username (master) and password (iAmReady) as default values so it runs self-contained. Marking a variable as **sensitive** masks its value in the designer and run details, but it does **not** remove the value from a flow export. In production, never store credentials in variable defaults. Use a secure credential store instead — see [Appendix E](#). The same credentials reappear in Steps 2, 4, and 5; treat every one of those as a placeholder for a securely retrieved secret.

The configured portal and browser input variables

Figure: The configured portal and browser input variables.

1. Select the **Subflows** dropdown menu in the upper left part of the screen. Then, select **+ New** subflow to create a subflow named **1_Prechecks**. Numbered prefixes make the intended execution order easier to understand.

Creating a new subflow

Figure: Creating a new subflow.

Adding a subflow

Figure: Adding a subflow.

The 1_Prechecks subflow

Figure: The 1_Prechecks subflow.

1. In the **actions pane** on the left side of the flow designer, search for the **On block error** action and add it to the flow by double-clicking it or dragging and dropping it into the 1_Prechecks Subflow.
 - In the **On block error** section, add the **If file exists** action and set the following values:
 - **If file:** - Doesn't exist
 - **File path:** - %ci_Edge_Path%
 - In the **If file exists** section, add the **Log Message** action and set the following values:
 - **Message:** - Edge Browser doesn't exist, hence stopping the flow.
 - **Log level:** - Error
 - In the **If file exists** section, add the **Throw custom error** action and set the following values:
 - **Error name:** - Edge Browser Issue.
 - **Error message:** - Edge Browser doesn't exist at the path %ci_Edge_Path%, hence stopping the flow
 - Under the **If file exists** section, insert an **Else** action, then place the **Log Message** action within **Else** and configure the following values:
 - **Message:** - Browser exists, hence starting the process
 - **Log level:** - Info

Once the configuration is complete, the subflow should look like this:

The completed 1_Prechecks browser-existence check

Figure: The completed 1_Prechecks browser-existence check.

1. Save the desktop flow by selecting **Save draft**.

Saving the desktop flow as a draft

Figure: Saving the desktop flow as a draft.

✓ **Expected result — checkpoint** ✓ The Provision Partner Users desktop flow is saved. ✓ The URL, username, password, and Edge path inputs exist.
✓ The 1_Prechecks Subflow runs without unexpected validation errors.

Step 2: Capture UI elements and sign in to the Northwind User Admin Portal

Step-by-step instructions

1. Create a Subflow named **2_Login**.

The 2_Login subflow tab

Figure: The 2_Login subflow tab.

1. Open <https://nwtua.z13.web.core.windows.net/> in Microsoft Edge so you can capture its UI elements.

The Northwind User Admin Portal sign-in screen

Figure: The Northwind User Admin Portal sign-in screen.

1. In the designer, open the **UI elements pane**, select **UI elements** and **Add UI element**.

Adding a UI element from the UI elements pane

Figure: Adding a UI element from the UI elements pane.

1. Point to the required control in the browser, and use **Ctrl+left** click to capture it.

- Capture **Codename** and click **Done**.

Capturing the Codename field

Figure: Capturing the Codename field.

- Capture **Password** and click **Done**.

Capturing the Password field

Figure: Capturing the Password field.

- Capture **Login** button and click **Done**.

Capturing the Login button

Figure: Capturing the Login button.

- Capture **Div 'loginErr'** directly below the login button and click **Done**.

Capturing the Div 'loginErr' element

Figure: Capturing the Div 'loginErr' element.

- Log in with **Codename** set to **master** and **Secret passphrase** set to **iAmReady**. A **new user record** form will appear — capture the **New User Record** header.

Capturing the New User Record header

Figure: Capturing the New User Record header.

1. Confirm that the captured UI elements controls appear in the **UI elements pane**, as shown below.

The captured login UI elements

Figure: The captured login UI elements.

1. Search for and add the actions shown below to the **2_Login Subflow**. The subflow wraps its actions in an **On block error** block named `Login`, split into two regions — `LoginSteps` and `LoginHandler`. Add the actions in order; you configure the error-handling behaviour (retry policy and go-to-label) later in Step 5.

Region: LoginSteps

1. **Launch new Microsoft Edge** — navigate to `%ci_NWUA_URL%` and store the instance into `fBrowser`.
2. **Wait for web page content** — wait for UI element **Input text 'u'** (the username field) to appear.
3. **Populate text field on web page** — populate **Input text 'u'** with `%ci_NWUA_username%` using emulated typing.
4. **Populate text field on web page** — populate **Input password 'p'** with `%ci_NWUA_Password%` using emulated typing.
5. **Press button on web page** — press **Button 'Login'**.

Region: LoginHandler

1. **Wait for web page content** — wait for UI element **Heading 1 'New User Record'** to appear.
2. **If web page contains** — if UI element **Heading 1 'New User Record'** exists on the web page:
 - **Log message** — level **Info**, message `Login is successful`.
 - **Exit subflow**.
3. **Label** — `Login Error`.
4. **Get details of element on web page** — get attribute **Own Text** of element **Div 'loginErr'** and store it into `fLoginErrorMsg`.
5. **Throw custom error** — error name `Login Error`, message `Login has failed, perform access troubleshooting %fLoginErrorMsg%`.

Use the screenshots below to confirm the completed subflow.

 **Tip** - Variables produced by actions are flow-scoped. Prefix their names with `f` so they are easy to distinguish from configurable inputs.

The 2_Login subflow actions

Figure: The 2_Login subflow actions.

The 2_Login subflow actions, continued

Figure: The 2_Login subflow actions, continued.

1. Run the **2_Login Subflow** and confirm that Microsoft Edge opens and the portal sign-in completes successfully.

Running the 2_Login subflow with default values

Figure: Running the 2_Login subflow with default values.

 Expected result — checkpoint

- ✓ Microsoft Edge opens the Northwind portal: **User Admin Portal**.

- ✓ The **2_Login Subflow** enters the workshop credentials.
- ✓ The portal displays the signed-in experience.
- ✓ The portal home page appears after sign-in.

Step 3: Read and validate user data from a CSV file

Step-by-step instructions

 **Goal** - Read user data from a CSV file and validate every required field before data entry.

Use the [Users CSV file](#) as the source for user entry. It intentionally contains extra spaces and invalid values so you can test the validation logic.

First Name	Last Name	Email	Phone	Address
Reginald	Cufflinks	reg.c@spy.co.uk	5550172394	7 Tuxedo Lane, Mayfair, London W1K 3QA
Svetlana	Borscht	sveta@spy.su	5550419827	12 Perpetual Winter Blvd, Apt. KGB, Moscow
Chadsworth	Fondue	chad@spy.su	5550936	3 Numbered- Account Alley, Geneva 1204
Guadalupe	Pistachio	lupe	5550287365	88 Hidden Compartment Way, Mexico City
Bartholomew	Trenchcoat	bart@spy.us	5550648193	1 Suspiciously Normal St, Langley, VA 22101

1. Create a **Subflow** named **3_User_Entry**.

Creating the 3_User_Entry subflow

Figure: Creating the 3_User_Entry subflow.

1. Before you add the Read action, get the source file onto the machine:
 1. Download [resources/users.csv](#) from the lab repository to the Windows machine.
 2. Note the full local path, for example `C:\Users\<username>\Downloads\users.csv`.
 3. You will enter that path in the **File path** field of the **Read from CSV file** action in the next step.
2. Add the **Read from CSV file** action. Set **File path** to the local path of `users.csv` from the previous step. Under **Advanced**, enable **Trim** fields and **First line** contains column names. Then rename the variable to **fCSVTable**.

The configured Read from CSV file action

Figure: The configured Read from CSV file action.

1. Add a **For each** action where **Value to Iterate** is %fCSVTable% and **Store into** is fCurrentItem.

Adding the For each action over fCSVTable

Figure: Adding the For each action over fCSVTable.

1. Inside the **For each** action, create the variables shown below and map each one to the corresponding value in the current CSV row.

Variable	Value
fFirstName	%fCurrentItem['First Name']%
fLastName	%fCurrentItem['Last Name']%
fEmail	%fCurrentItem['Email']%
fPhone	%fCurrentItem['Phone']%
fAddress	%fCurrentItem['Address']%

Mapping each field to the current CSV row

Figure: Mapping each field to the current CSV row.

1. To validate whether the email value %fEmail% has a valid structure, add a **Parse text** action and apply the regular expression below:

```
\b[\w\.-]+@[\w\.-]+\.\w{2,4}\b
```

Validating the email with a Parse text action

Figure: Validating the email with a Parse text action.

1. Add a **Set variable** action, then configure a condition to check that all required values are present, the phone number is exactly 10 characters long, and the email validation returns a match.

```
%IsNotEmpty(fFirstName) AND IsNotEmpty(fLastName) AND IsNotEmpty(fEmail) AND  
IsNotEmpty(fPhone) AND IsNotEmpty(fAddress) AND (fPhone.Length = 10) AND  
(fPosition = 0)%
```

Setting the fValidation condition

Figure: Setting the fValidation condition.

1. Compare your completed **3_User_Entry** Subflow with the example below.

The completed 3_User_Entry subflow

Figure: The completed 3_User_Entry subflow.

1. Run the **Subflow** with the workshop CSV file and confirm that valid and invalid records follow the expected branches.

✓ Expected result — checkpoint

- ✓ The workshop CSV loads into a data table.
- ✓ Required fields, phone length, and email structure are checked.
- ✓ Valid and invalid rows follow different branches.

Step 4: Enter validated user data in the Northwind User Admin Portal

Step-by-step instructions

 **Goal** - Populate and submit the Northwind User Admin Portal form with validated data.

1. Open <https://nwtua.z13.web.core.windows.net/> in Microsoft Edge so you can capture its UI elements. Log in with **Codename** set to **master** and **Secret passphrase** set to **iAmReady**. A **new user record** form will appear.

The Northwind portal sign-in screen

Figure: The Northwind portal sign-in screen.

1. In the designer, open the **UI elements pane**, select **UI elements** and **Add UI element**.

The New User Record form

Figure: The New User Record form.

1. Point to the required control in the browser, and use **Ctrl+left** click to capture it.
 - Capture **First Name** and click Done.

Capturing the First Name field

Figure: Capturing the First Name field.

- Capture **Last Name** and click Done.

Capturing the Last Name field

Figure: Capturing the Last Name field.

- Capture **Email** and click Done.

Capturing the Email field

Figure: Capturing the Email field.

- Capture **Phone** and click Done.

Capturing the Phone field

Figure: Capturing the Phone field.

- Capture **Address** and click Done.

Capturing the Address field

Figure: Capturing the Address field.

- Capture **Submit** and click Done.

Capturing the Submit button

Figure: Capturing the Submit button.

1. Capture the UI elements for the **First Name**, **Last Name**, **Email**, **Phone**, **Address**, and **Submit** controls.

The captured form UI elements

Figure: The captured form UI elements.

1. Enter any values for **First Name**, **Last Name**, **Email**, **Phone**, and **Address**, then click **Submit**. When the confirmation popup appears:
 - Capture the paragraph **Are you sure you want to submit the dataset?** and click Done.

Capturing the submission confirmation popup

Figure: Capturing the submission confirmation popup.

- Capture the button **OK** and click Done.

The confirmation popup OK and Cancel buttons

Figure: The confirmation popup OK and Cancel buttons.

1. Confirm that the captured UI elements controls appear in the **UI elements pane**, as shown below:

The captured form and confirmation UI elements

Figure: The captured form and confirmation UI elements.

1. Insert the actions below into the **3_User_Entry** subflow, right after the **Set variable** action: **fValidation**.
 - Add the conditional action **If** where the **fValidation** variable is equal to **True**.

The full list of captured UI elements

Figure: The full list of captured UI elements.

- Within the **If** block, add the **Populate text field in web page** action, then for each field (first name, last name, email, phone, address), choose the saved UI element and map it to the matching validated variable as shown below.

Configuring a Populate text field action

Figure: Configuring a Populate text field action.

1. Compare your completed **3_User_Entry** Subflow condition **If** with the example below.

The If fValidation block that populates and submits the form

Figure: The If fValidation block that populates and submits the form.

1. Add the actions shown below to handle the submission confirmation popup:
 1. **Wait** — 3 seconds.
 2. **If web page contains** — if UI element **Paragraph 'Are you ... after submission.'** exists on the web page:
 - **Press button on web page** — press **Button 'OK'**.
 - **Wait** — 3 seconds.

Use the screenshot below to confirm the completed sequence.

Handling the submission confirmation

Figure: Handling the submission confirmation.

✓ Expected result — checkpoint

- ✓ The form fields receive values from the validated CSV row.
- ✓ Submit triggers the expected confirmation or status message.
- ✓ The flow can continue to the next record.

Step 5: Add error handling, flow control, logging, and resilient UI-element handling

Step-by-step instructions

 **Goal** - Add retries, logging, flow control, and resilient UI-element handling.

1. Open <https://nwtua.z13.web.core.windows.net/> in Microsoft Edge so you can capture its UI elements. Log in with **Codename** set to **master** and **Secret passphrase** set to **iAmReady**. A **new user record** form will appear.

The Northwind portal sign-in screen

Figure: The Northwind portal sign-in screen.

1. In the designer, open the **UI elements pane**, select **UI elements** and **Add UI element**.

The New User Record form

Figure: The New User Record form.

1. Select **Add UI element**, point to the required control in the browser, and use **Ctrl+left** click to capture it.
 - Enter any values for **First Name**, **Last Name**, **Email**, **Phone**, and **Address**, then click **Submit**. When the confirmation popup appears, capture the paragraph **"Are you sure you want to submit the dataset?"** and the **OK** button. A toast message appears after a successful submission — capture it so the flow can record the correct outcome.

Capturing the New User Record element

Figure: Capturing the New User Record element.

1. The **Status** UI element is now in place.

Capturing the Div 'User record' element

Figure: Capturing the Div 'User record' element.

1. Add the actions shown below to process each record and write useful log messages:
 1. **If web page contains** — if text '**User record submitted and form cleared.**' exists on the web page:
 - **Log message** — level **Info**, message **Successful Entry** - followed by `%fFirstName%`, `%fLastName%`, `%fEmail%`, `%fPhone%`, `%fAddress%`.
 2. **Else:**
 - **Log message** — level **Error**, message **Invalid Entry** - followed by `%fFirstName%`, `%fLastName%`, `%fEmail%`, `%fPhone%`, `%fAddress%`.

Use the screenshot below to confirm the completed logging branch.

Logging successful and invalid entries

Figure: Logging successful and invalid entries.

1. Each action has **On error settings**. In the **2_Login** subflow, open the **Wait for web page content** action (the one that waits for **Input text 'u'**) and configure its retry policy with these values:
 - **Retry policy:** Fixed
 - **Times:** 2
 - **Interval:** 10 seconds

The same retry values (Fixed, 2 times, 10-second interval) apply wherever this lab configures a retry policy.

Configuring the retry policy in On error settings

Figure: Configuring the retry policy in On error settings.

1. In the **2_Login** subflow, use labels and flow-control actions to direct execution when an action fails. Edit the **If web page contains** action in the **LoginHandler** region and set its **On error** settings:
 - **Retry policy:** Fixed — **Times** 2, **Interval** 10 seconds.
 - Under **All errors (default)**, select **Continue flow run**.
 - **Exception handling mode:** Go to label.
 - **Select label:** `Login Error` (the label added earlier in the LoginHandler region).

Directing execution to the Login Error label on failure

Figure: Directing execution to the Login Error label on failure.

1. Configure the **On block error** action named `Login` in the **2_Login** subflow:
 - **Retry policy:** Fixed — **Times** 2, **Interval** 10 seconds.
 - Under **All errors (default)**, select **Throw error**.
 - Turn **Handle flow terminating errors** on.

Configuring the On block error action

Figure: Configuring the On block error action.

1. For web UI actions, enable self-healing preview where appropriate to improve resilience when selectors change. In **2_Login** and **3_User_Entry** subflows, edit **Populate text field on web page** action as shown below.

Enabling self-healing on a Populate text field action

Figure: Enabling self-healing on a Populate text field action.

1. If execution fails on the **phone-number field**, inspect the captured UI element. Its generated ID changes dynamically and should not be the primary selector. Retain the stable name attribute, and remove or relax the changing ID attribute, as shown below.

Relaxing the dynamic ID on the phone-number selector

Figure: Relaxing the dynamic ID on the phone-number selector.

Where appropriate, configure an image as a fallback when selector-based UI-element lookup fails.

1. In Main, call the subflows in the sequence shown below.

Calling the subflows in order from Main

Figure: Calling the subflows in order from Main.

1. Save the flow, run **Main**, and confirm that the end-to-end flow runs successfully from start to finish.

✓ Expected result — checkpoint

- ✓ Successful and failed records produce useful log messages.
- ✓ Transient web failures retry or follow the configured error path.
- ✓ The phone selector uses a stable attribute, and the main flow runs end to end.

Step 6: Publish a reusable UI collection

Step-by-step instructions

 **Goal** - Publish reusable UI elements as a UI collection.

1. UI collections let multiple desktop flows reuse the same published UI elements. When an element changes, you update the collection once instead of updating every flow. **Creating a UI collection requires a premium license.**
2. Rename UI elements so their purpose is clear before publishing the collection.

Renaming UI elements before publishing

Figure: Renaming UI elements before publishing.

1. Select the required UI elements by choosing their top-level web page container and click **Publish as new collection**.

Selecting the UI elements to publish

Figure: Selecting the UI elements to publish.

1. Provide a name for the new collection. If you associated any or all of the selected UI elements with UI or web automation actions in your desktop flow, you can also check the 'Auto-update' option below the collection name field. This automatically updates the related actions, ensuring they reference the newly established counterparts in the collection, rather than the UI elements previously accessible only within this flow.

To make any required adjustments to a collection, that collection needs to be imported to a desktop flow, so that you can access the collection's contents in the flow designer. In addition, you need to be an Owner or have at least Co-owner rights on that collection to be able to modify it.

Publishing the UI elements as a new collection

Figure: Publishing the UI elements as a new collection.

1. Confirm that the published UI collection appears as shown below.

The published UI collection

Figure: The published UI collection.

✓ Expected result — checkpoint

- ✓ The UI collection is published with a descriptive name.
- ✓ The current flow references the published collection.
- ✓ The reusable elements are available for other desktop flows.

Step 7 (Optional): Trigger the desktop flow from a cloud flow

Step-by-step instructions

 **Goal** - Run the desktop flow on demand from a cloud flow instead of starting it manually.

This optional step shows how a cloud flow can trigger the automation you built. It is not required to complete the lab. For local attended runs and picture-in-picture, see [Appendix B: Run a desktop flow in different modes](#).

1. Go to the [Power Automate portal](#), select **Create**, and then select **Instant cloud flow**.

Creating an instant cloud flow

Figure: Creating an instant cloud flow.

1. After the trigger, select the **+** button to add an action, and then search for **desktop flow**.

Searching for the desktop flow action

Figure: Searching for the desktop flow action.

1. Select **Run a flow built with Power Automate for desktop**.
2. In **Desktop flow**, select **Provision Partner Users**.
3. Select the connection that defines the machine or machine group where the flow runs and the credentials used to sign in. If no connection exists, create one.

Configuring the desktop flow action

Figure: Configuring the desktop flow action.

1. In **Run mode**, select **Attended**.
2. Save and publish the cloud flow.
3. Run the cloud flow and confirm that the desktop flow executes in attended mode.

Note: Unattended runs require the Unattended RPA add-on for the environment, a target machine or machine group, and stored credentials. No user needs to be signed in, but the machine must be powered on with no interactive session blocking it.

1. After the attended run succeeds, change **Run mode** to **Unattended**, save the cloud flow, and run it again.
2. Confirm that the desktop flow completes successfully in unattended mode.

✓ **Expected result — checkpoint** ✓ A cloud flow triggers the **Provision Partner Users** desktop flow. ✓ The desktop flow runs in attended mode and completes end to end.

Lab completion

🎉 Congratulations! You have built a complete attended RPA solution that signs in to a legacy web application, reads and validates CSV data, submits valid records, handles common failures, writes execution logs, and publishes reusable UI elements.

Summary & best practices

- Move credentials to an approved secret store (Azure Key Vault, Power Automate credentials, or CyberArk) before any production use.
- Test with a wider range of data, including edge cases, before go-live.
- Define monitoring and support ownership for the desktop flow.
- Consider hosted machines for controlled unattended execution at scale.
- Update the UI collection any time a selector changes — don't patch individual flows.

Recommended next step

Continue to [Module 7: Workflow](#) to create an agent flow that a Copilot Studio agent calls as a tool, mid-conversation, to get a grounded answer back.

Appendix A: Register a machine with Power Automate

If no machines appear in the **Machines** list, register one. Machine registration connects a physical or virtual Windows machine directly to your Power Platform environment without requiring a gateway. After registration, the machine is available for desktop flow runs.

Prerequisites

- A physical or virtual Windows machine where you can sign in as an administrator.
- The latest version of Power Automate for desktop installed on that machine.
- A Power Automate license that permits machine management.

- The **Environment Maker** role, or an equivalent role, in the target environment.

Register the machine

1. On the target machine, download and install Power Automate for desktop from the [Power Automate portal](#) or [Download Power Automate for desktop](#). Run the installer and complete the setup. The installer also installs the Power Automate machine runtime.

Installing Power Automate for desktop from the portal

Figure: Installing Power Automate for desktop from the portal.

2. Launch Power Automate for desktop and sign in with your work or school account.

Signing in to Power Automate for desktop

Figure: Signing in to Power Automate for desktop.

1. From the Windows Start menu, search for and open **Power Automate machine runtime**.

Opening the Power Automate machine runtime

Figure: Opening the Power Automate machine runtime.

1. In the machine runtime app, select **Machine settings**, and then select **Register a new machine**.

Registering a new machine from the machine runtime

Figure: Registering a new machine from the machine runtime.

1. Select the target environment from the dropdown. Use the same environment where you created the desktop flow. Registration starts automatically.

Selecting the environment during machine registration

Figure: Selecting the environment during machine registration.

1. Wait for registration to finish. Confirm that the runtime app shows the machine name, its environment, and a **Connected** status.

The registered machine showing Connected status

Figure: The registered machine showing Connected status.

1. In the [Power Automate portal](#), confirm that the correct environment is selected. Then go to **Monitor > Machines**.

The Machines page in the Power Automate portal

Figure: The Machines page in the Power Automate portal.

1. Confirm that the newly registered machine appears in the list. You can now use it to run the desktop flow in this lab.

Machine registration notes

- A machine can be configured for either computer use or RPA desktop flows at one time, but not both simultaneously. Change the mode in the machine settings when needed.
- Keep the machine powered on and signed in when runs need to execute.

- If the machine does not appear, verify that you registered it in the correct environment and that your account has machine-management privileges.

Appendix B: Run a desktop flow in different modes

Run mode	Triggered from	License required
Local attended	Power Automate for desktop. The desktop flow runs interactively on your own machine.	Windows 11 includes free Power Automate for desktop runs. Desktop flows that use premium features require Power Automate Premium with attended RPA.
Attended (cloud)	A cloud flow calls the desktop flow as an action while you are signed in to the machine.	Power Automate Premium with attended RPA.
Unattended (cloud)	A cloud flow calls the desktop flow as an action with no user signed in.	An Unattended RPA add-on in addition to a Premium or Process plan.

Cloud flows let you trigger the desktop flow instantly, on a schedule, or from another event.

Run the desktop flow locally in attended mode

1. On the machine, open Power Automate for desktop and sign in with your work or school account.
2. Select the **Provision Partner Users** desktop flow, and then select **Run**. The flow runs interactively on your machine so you can watch each action.

Run the desktop flow from a cloud flow

A cloud flow can trigger the desktop flow in attended or unattended mode. This is covered as an optional lab step — see [Step 7 \(Optional\): Trigger the desktop flow from a cloud flow](#).

Run attended flows in picture-in-picture

Picture-in-picture (PiP) runs an attended automation in a separate desktop environment, allowing you to continue working on your main desktop while the flow runs. PiP is available for attended runs only; it is not supported for unattended runs.

When a cloud flow triggers a desktop flow in PiP, only child session mode is supported. In the **Run a flow built with Power Automate for desktop** action, set **Run mode** to **Attended**, and then set **Attended mode** to **Picture-in-picture**.

PiP mode	What it is	Best for
Child session	A full secondary Windows session with its own UI, processes, and application state.	UI-heavy automation that uses the mouse, keyboard, Office, PDFs, or legacy Win32 applications; interactive step-by-step debugging.
Virtual desktop (preview)	An isolated virtual desktop in the same user session, with a clean environment for each run.	Stability, isolation, compliance, and UI Automation-based flows.

Enable PiP on the machine with administrator rights. You can enable it manually, through the MSI installer option, during a silent installation, or with `PAD.ChildSession.Installer.Host.exe`. Start a PiP run from the console by selecting **More actions > Run in picture-in-picture**, configure it in the flow properties, or debug with **Debug > Enable picture-in-picture mode** in the designer.

Learn more: [Run a desktop flow in picture-in-picture](#).

Appendix C: Monitor runs with Desktop Flow Runs

1. Go to the [Power Automate portal](#), select **More > Discover all**, scroll to the **Monitor** section, and then select **Desktop flow runs**.

Opening Desktop flow runs in Monitor

Figure: Opening Desktop flow runs in Monitor.

1. Review the runs from the local attended, cloud attended, and cloud unattended execution modes.

The Desktop Flow Runs page

Figure: The Desktop Flow Runs page.

1. Select a run to open its execution details.

The details show the status, start time, duration, run mode, and trigger. They also show each action, captured screenshots, and error messages for troubleshooting. Use the filters to narrow the list by status or date.

Desktop flow run details

Figure: Desktop flow run details.

Appendix D: Monitor runs with Automation Center

1. Go to the [Power Automate portal](#), select **Automation Center**, and then select **Runs**, and then select **Current Desktop flow runs**. Explore all the options inside the runs.

The Automation Center Runs page

Figure: The Automation Center Runs page.

1. Explore the other Automation Center views.

Automation Center provides a unified view of cloud and desktop flows, including run history, trends, error insights, and recommendations. Filter by status, flow, or date to investigate failures.

Appendix E: Credentials and secrets in desktop flows — mechanisms explained

Desktop flows deal with two credential contexts: signing into the machine (the connection) and secrets used inside the flow (app/website logins and service/API calls). The mechanisms below cover both, plus the vaults that back them. They are layered — not interchangeable — and Azure Key Vault is the secure foundation the strongest options build on.



The **machine connection** below is also a **cloud-flow** concept: a cloud flow authenticates to the target machine through a desktop-flow connection when it calls *Run a flow built with Power*

Automate for desktop. For the cloud-flow side (connections, environment-variable secrets, Key Vault connector, and Secure inputs/outputs), see [Module 1 · Appendix C: Secret handling in cloud flows](#).

#	Mechanism	Used for	Secure storage	Rotation-friendly	Shareable?	Scope	Best for
1	Saved credential (Credentials page)	Machine access (connection)	Azure Key Vault- or CyberArk-backed	✅ High — rotate once, all uses update	✅ Yes — shares a reference, not the secret	Environment (Connection / Desktop flow / Network)	Governed, reusable team sign-in — default
2	Manual entry (username + password)	Machine access (connection)	Encrypted on the connection	❌ Low — edit each connection	❌ No — connections aren't shareable	Connection (single)	A quick, single connection
3	Attended sign-in (connect with sign-in)	Machine access (attended only)	No stored secret — user's session / MFA	✅ N/A — nothing to rotate	❌ No — each user signs in as themselves	Per user / run	Orgs that forbid stored passwords
4	Connector connection (cloud connectors in the flow)	Calling a service / API from the flow	Managed connection (OAuth / API key, user-specific)	✅ High — OAuth auto-refresh; connection references ease updates	⚠️ Via embedded connection references (co-owners); run-only users bring their own	Connection / connection reference (ALM)	Structured API integration vs UI automation
5	Cloud flow + Key Vault connector → desktop flow input	In-flow app / website secrets	Value in Azure Key Vault, fetched at run time	✅ High — rotate in Key Vault	⚠️ With the cloud flow (its Key Vault connection is shared)	Flow run (secret in Key Vault)	App / site logins used inside the flow
6	Environment variables (Secret type)	In-flow secrets via the triggering cloud flow	Value in Azure Key Vault; Dataverse holds a pointer	✅ High — rotate in Key Vault; all envs update	✅ Yes — environment-wide (in the solution)	Environment / solution (ALM)	Secrets / config across dev/ test/prod
7	Power Automate secret variables	Masking secrets within the flow at runtime	Not exposed in logs / UI; sourced from a connector or input	⚠️ Depends on the source (e.g., Key Vault)	⚠️ Travels with the flow definition	Flow run	Keeping a fetched secret masked mid-flow
8	CyberArk		Enterprise vault (CCP)	✅ High — central	✅ Yes — via the shared		

#	Mechanism	Used for	Secure storage	Rotation-friendly	Shareable?	Scope	Best for
		Machine access / credentials	retrieval at runtime)	rotation in CyberArk	Credential (reference)	Environment (credential) / external vault	Orgs standardised on CyberArk
9	Azure Key Vault	The foundation for the above	Enterprise store (RBAC, audit, HSM option)	★ Highest — versioning + rotation policies	🟡 Via RBAC — grant identities / groups	Tenant / subscription	The secure backing store the others build on

Recommended credential pattern

- For machine access, use a saved credential backed by Azure Key Vault or CyberArk instead of manual entry.
- For an application secret used inside a flow, retrieve it from Azure Key Vault or a secret environment variable, pass it as a secure input, and keep it masked with a secret variable.
- For service and API calls, use connector connections and connection references. Co-owners can use embedded references; run-only users can provide their own connections.
- Use a separate Azure Key Vault for each environment with least-privilege role-based access and rotation policies. Promote references across development, test, and production, never the secret values.

Note: Features, availability, and licensing change over time. Refer to the linked Microsoft Learn documentation for current product behavior.

Further reading: [Manage desktop flow connections](#), [share desktop flows that contain connector actions](#), [use a connection reference in a solution](#), and [automate Key Vault secret rotation](#).

Appendix F: Machine connection authentication methods

A machine connection defines how Power Automate signs in to the machine that runs a desktop flow. Choose the authentication method based on your security policy and whether the flow must run unattended.

Method	How it authenticates	Secret stored?	Supported runs	Rotation
Select credential	Uses a reusable environment credential backed by Azure Key Vault or CyberArk.	The secret remains in the external vault and is referenced rather than exposed.	Attended and unattended.	Rotate once centrally; every connection that uses the credential is updated.
Enter username and password	Uses a domain account such as DOMAIN\User or user@domain.com, or a local account such as	The password is encrypted on the connection.	Attended and unattended.	Update each connection manually.

Method	How it authenticates	Secret stored?	Supported runs	Rotation
	MACHINE\User or local\User .			
Connect with sign-in	Uses a Microsoft Entra ID access and refresh token scoped to desktop-flow execution and managed by Power Platform.	No password is stored.	Attended only. Unattended runs fail.	No password rotation is required; the token is managed automatically.

Connect with username and password

You can provide the machine's device account in either of these ways:

- **Select credential (recommended):** Select an existing environment credential backed by Azure Key Vault or CyberArk. If one does not exist, create a credential and associate it with a vault secret.
- **Enter username and password:** Type the account and password directly on the connection. This is simple, but the secret is tied to that connection and must be updated there whenever it changes.

Both options support attended and unattended runs.

Connect with sign-in

Use this passwordless option when organizational policy does not permit stored usernames and passwords. In the **Connect** dropdown, select **Connect with sign-in**, select the target machine or group, select **Sign in**, and choose a Microsoft Entra account. Power Platform creates the connection automatically.

Prerequisites and limitations:

- The Microsoft Entra user must be in the same tenant as the environment and must be permitted to open an interactive Windows session.
- The machine or group must be Microsoft Entra joined or Active Directory domain joined. Active Directory-only targets require the Power Platform tenant to be allowlisted.
- The tenant must use modern authentication.
- This method supports attended runs only, and the queue time is limited to one hour.
- Desktop flow connection sharing is limited to service-principal users with **Can use** or **Can edit** access.
- Unattended runs also require the Unattended RPA add-on and a machine or machine group.

Learn more: [Manage desktop flow connections](#) and [allowlist tenants for connect with sign-in](#).

Appendix G: Keep a desktop flow clean

Cluttered flows are harder to read, share, and maintain. Before publishing, remove unused content and document what remains.

Remove unused UI elements

In the **UI elements** pane, select the ellipsis next to **Sort**, and then select **Remove unused UI elements**. This removes every UI element that is not referenced by an action. To review an individual element, right-click it and select **Find usages**, **Rename**, or **Delete**.

Remove unused images

In the **Images** pane, select the ellipsis next to **Capture image**, and then select **Remove unused images**. When you run this command inside a folder, it removes only that folder's unused images. **Find usages**, **Rename**, and **Delete** are also available for individual images.

Remove unused and disabled actions

Delete disabled actions, temporary test steps, and outdated comment placeholders before publishing. Disabled actions remain in the flow definition but never run, which adds noise and maintenance risk.

Clean up variables

Power Automate for desktop does not provide a **Remove unused** command for variables. Remove them manually: delete obsolete **Set variable** actions and use **Find usages** before removing a variable. Use clear names and avoid reserved keywords such as `action`, `loop`, and `if`.

Comment effectively

- Use the **Comment** action under **Flow control** to explain why the flow uses a particular approach, not what each visible action does.
- Add a short comment at the start of each subflow or region that states its purpose, inputs, and outputs.
- Use **Region** and **End region** to group related actions, and collapse regions to keep large flows readable.
- Update or remove comments whenever the logic changes.
- Never include secrets in comments.
- Prefer descriptive names for variables, subflows, and UI elements so fewer comments are necessary.

Learn more: [Automate using UI elements](#), [images](#), [variable actions](#), and [flow control actions](#).

Appendix H: Debug a desktop flow in the designer

Use these tools to debug interactively in the Power Automate for desktop flow designer.

Tool	How to use it	Use it to
Run or pause	Select Run or press <code>F5</code> . Press <code>Ctrl+Pause</code> to pause, and select Run to resume.	Inspect the flow state at any point.
Run next action	Press <code>F10</code> to run one action and pause again.	Step through the flow one action at a time.
Breakpoints	Select the area to the left of an action's order number to add a red breakpoint; select it again to remove it.	Pause at a chosen action.
Stepping mode	Select Debug > Stepping mode . Use <code>F11</code> to step over and <code>Shift+F11</code> to step out.	Run or skip a subflow, or return from a subflow to its caller.
Run from here	Right-click an action and select Run from here .	Start at a chosen action without running earlier actions.
Run delay	Set a delay in milliseconds in the status bar.	Slow the flow so you can observe its behavior.

Tool	How to use it	Use it to
Variables pane	Open the pane while the flow is paused.	Inspect or modify variable values during a run.
Errors pane	Select the error count in the status bar; actions with errors are highlighted.	Find and navigate to failing actions.
Debug in PiP	Select Debug > Enable picture-in-picture mode .	Debug without occupying your main desktop.

For new logic, use **Run next action** with the **Variables** pane to verify values. Combine breakpoints with **Run from here** to retest only the section you changed. Add a short run delay to investigate UI timing, and resolve every item in the **Errors** pane before publishing.

Learn more: [Debugging a desktop flow](#) and [Errors pane](#).

Appendix I: Safe stop versus Stop

Power Automate for desktop provides two ways to halt a running flow, and they behave differently.

	Stop	Safe stop
Behavior	Ends the flow immediately.	Finishes the current action, runs a cleanup block, and then stops.
Data integrity	May leave applications, files, or systems in an inconsistent state.	Supports a graceful shutdown that preserves integrity.
Flow authoring	Requires no additional actions.	Requires If safe stop is requested actions at logical checkpoints.
Triggered from	Stop in the designer or console, or Shift+F5 .	The portal run details, the console's Stop menu or context menu, or the designer's Stop menu.
Best for	Development interruptions where cleanup does not matter.	Production flows and any automation that modifies data, uses files, or holds open connections.

How safe stop works

Insert **If safe stop is requested** at logical checkpoints. When a safe stop is requested, the flow completes the current action, reaches the next checkpoint, and runs the actions inside the block. Use that block to save files, close connections, and log status.

Set **Stop the flow** to **True** to stop after the block. If it is **False**, add a **Stop flow** action to the block yourself.

Key limitations:

- Request the safe stop before the flow passes a checkpoint. Otherwise, it waits for the next checkpoint.
- Safe stop is not a pause-and-resume mechanism.
- Safe stop works on parent flows only. Requesting it from a child flow returns a run-not-found error.

Use safe stop for any flow that writes data or holds resources. Reserve plain **Stop** for temporary debugging where cleanup does not matter. Place **If safe stop is requested** wherever an abrupt halt could corrupt state.

Learn more: [Safe stop](#) and [flow control actions](#).

Appendix J: Names used in this lab

This lab uses a consistent naming convention. The **f** prefix marks variables the flow produces; the **ci_** prefix — *configurable input* — marks values supplied to the flow from outside rather than produced within it. Each name is introduced the first time it is used in a step; this table is a full reference.

Component	Name
Desktop flow	Provision Partner Users
Subflows	1_Prechecks, 2_Login, 3_User_Entry
Input variables	ci_NWUA_URL, ci_NWUA_username, ci_NWUA_Password, ci_Edge_Path
Output variables	fAddress, fBrowser, fCSVTable, fCurrentItem, fEmail, fFirstName, fLastName, fLoginErrorMsg, fPhone, fPosition, fValidation, fMatch
Contact file	users.csv
UI elements collection	Northwind User Admin Portal